

Problema Lab 4: Optimizarea performanței și bune practici

Obiectiv

Aplicați tehnicile de optimizare a performanței învățate pe parcursul cursului, identificați și rezolvați blocajele comune de performanță și implementați pattern-uri de acces la baza de date gata pentru producție.

Obiective de Învățare

- Identificarea și rezolvarea problemei N+1 query
- Înțelegerea impactului indexării asupra performanței interogărilor
- Implementarea strategiilor eficiente de paginare
- Adăugarea straturi de cache pentru date accesate frecvent
- Optimizarea operațiilor în masă
- Aplicarea best practices pentru accesul la baza de date

Cerințe

1. Problema N+1 Query

Sarcina A: Demonstrați Problema

Creați un scenariu care prezintă problema N+1 query:

```
// Rău: N+1 interogări
List<Department> departments = entityManager
    .createQuery("SELECT d FROM Department d", Department.class)
    .getResultList(); // 1 interogare

for (Department dept : departments) {
    List<Employee> employees = dept.getEmployees(); // N interogări
    System.out.println(dept.getName() + " are " + employees.size() + " angajați");
}
```

Cerințe:

- Activați logging-ul SQL pentru a arăta toate interogările executate
- Numărați numărul total de interogări
- Măsurați timpul de execuție
- Documentați cu cel puțin 10 înregistrări părinte

Sarcina B: Rezolvați cu Eager Loading

Rezolvați problema folosind JOIN FETCH sau Include():

Java (JPQL):

```
List<Department> departments = entityManager
    .createQuery("SELECT DISTINCT d FROM Department d LEFT JOIN FETCH d.employees", Department.class)
    .getResultList(); // doar 1 interogare
```

C# (LINQ):

```
var departments = context.Departments
    .Include(d => d.Employees)
    .ToList(); // doar 1 interogare
```

Comparație Necesară:

- Arătați log-urile SQL înainte și după
- Comparați timpii de execuție
- Documentați reducerea numărului de interogări

2. Analiza Performanței Indexării

Sarcina A: Benchmark Fără Index-uri

Creați un set de date cu cel puțin 10.000 de înregistrări de angajați și măsurați performanța interogărilor:

```
-- Interogare 1: Căutare după email
SELECT * FROM employees WHERE email = 'test@example.com';

-- Interogare 2: Căutare după departament
SELECT * FROM employees WHERE department_id = 5;

-- Interogare 3: Interogare interval pe salariu
SELECT * FROM employees WHERE salary BETWEEN 50000 AND 80000;

-- Interogare 4: Căutare multi-coloană
SELECT * FROM employees
WHERE department_id = 5 AND salary > 60000;
```

Cerințe:

- Executați fiecare interogare de 100 de ori și calculați timpul mediu de execuție
- Folosiți EXPLAIN ANALYZE din PostgreSQL pentru a arăta planurile de interogare
- Documentați scanările secvențiale vs scanările cu index

Sarcina B: Adăugați Index-uri Strategice

Creați migrări pentru a adăuga index-uri corespunzătoare:

```
CREATE INDEX idx_employees_email ON employees(email);
CREATE INDEX idx_employees_department_id ON employees(department_id);
```

```
CREATE INDEX idx_employees_salary ON employees(salary);
CREATE INDEX idx_employees_dept_salary ON employees(department_id, salary);
```

Sarcina C: Re-benchmark Cu Index-uri

- Re-rulați toate interogările din Sarcina A
- Comparați timpii de execuție înainte și după
- Arătați output-ul EXPLAIN ANALYZE demonstrând utilizarea index-urilor
- Creați un tabel/grafic de comparație

Tabel Rezultate Așteptate:

Interogare	Fără Index (ms)	Cu Index (ms)	Îmbunătățire
Căutare email			
Căutare departament			
Interval salariu			
Multi-coloană			

3. Implementare Paginare

Implementați două strategii de paginare și comparați performanța lor.

Strategia A: Paginare Bazată pe Offset (LIMIT/OFFSET)

```
public Page<Employee> getEmployeesPage(int pageNumber, int pageSize) {
    int offset = pageNumber * pageSize;

    List<Employee> employees = entityManager
        .createQuery("SELECT e FROM Employee e ORDER BY e.id", Employee.class)
        .setFirstResult(offset)
        .setMaxResults(pageSize)
        .getResultList();

    Long total = entityManager
        .createQuery("SELECT COUNT(e) FROM Employee e", Long.class)
        .getSingleResult();

    return new Page<>(employees, pageNumber, pageSize, total);
}
```

Strategia B: Paginare Bazată pe Cursor (Keyset)

```
public Page<Employee> getEmployeesAfter(Long lastId, int pageSize) {
    List<Employee> employees = entityManager
        .createQuery("SELECT e FROM Employee e WHERE e.id > :lastId ORDER BY e.id", Employee.class)
        .setParameter("lastId", lastId)
        .setMaxResults(pageSize)
        .getResultList();
}
```

```

        return new Page<>(employees, pageSize);
    }

```

Cerințe:

- Implementați ambele strategii în UI-ul aplicației
- Testați cu set de date de 10.000+ înregistrări
- Măsurați performanța pentru:
 - Prima pagină
 - Pagina din mijloc (pagina 50 din 100)
 - Ultima pagină
- Comparați încărcarea bazei de date (EXPLAIN ANALYZE)
- Documentați când fiecare strategie este potrivită

Cerințe UI:

- Butoane de navigare Anterior/Următor
- Afișare număr pagină
- Selector dimensiune pagină (10, 25, 50, 100)
- Număr total de înregistrări

4. Implementare Caching

Adăugați un strat de cache pentru datele read-only accesate frecvent.

Java - EhCache:

```

@Cacheable("departments")
public Department getDepartmentById(Long id) {
    return entityManager.find(Department.class, id);
}

@CacheEvict(value = "departments", key = "#department.id")
public void updateDepartment(Department department) {
    entityManager.merge(department);
}

```

C# - Memory Cache:

```

private readonly IMemoryCache _cache;

public Department GetDepartmentById(int id)
{
    return _cache.GetOrCreate($"dept_{id}", entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5);
        return context.Departments.Find(id);
    });
}

```

Cerințe:

- Cache-uiți departamentele (entități părinte)
- Implementați invalidarea cache-ului la actualizări
- Măsurați îmbunătățirea performanței:
 - Primul apel (cache miss)
 - Apelurile următoare (cache hit)
- Adăugați afișarea statisticilor cache (hit rate, miss rate)

Demonstrați:

- Cache miss: prima încărcare din baza de date
- Cache hit: încărcările ulterioare din cache
- Evicțiunea cache-ului: actualizarea declanșează reîmprospătarea cache-ului
- Expirarea TTL: invalidarea automată a cache-ului

5. Optimizarea Operațiilor în Masă

Sarcină: Optimizați Actualizarea în Masă

Comparați performanța diferitelor abordări de actualizare în masă:

Abordarea 1: Actualizări Individuale

```
List<Employee> employees = getEmployeesInDepartment(deptId);
for (Employee emp : employees) {
    emp.setSalary(emp.getSalary() * 1.1); // Mărire 10%
    entityManager.merge(emp);
}
```

Abordarea 2: Interogare Actualizare în Masă

```
entityManager.createQuery(
    "UPDATE Employee e SET e.salary = e.salary * 1.1 WHERE e.department.id = :deptId")
    .setParameter("deptId", deptId)
    .executeUpdate();
```

Abordarea 3: Actualizări Batch

```
List<Employee> employees = getEmployeesInDepartment(deptId);
int batchSize = 50;
for (int i = 0; i < employees.size(); i++) {
    Employee emp = employees.get(i);
    emp.setSalary(emp.getSalary() * 1.1);
    entityManager.merge(emp);

    if (i % batchSize == 0 && i > 0) {
        entityManager.flush();
        entityManager.clear();
    }
}
```

```
}
```

Cerințe:

- Testați cu 1000+ înregistrări
- Măsurați timpul de execuție pentru fiecare abordare
- Documentați care abordare este cea mai bună și de ce
- Considerați compromisurile (simplitate vs performanță vs flexibilitate)

6. Caching-ul Prepared Statements

Demonstrați beneficiul reutilizării prepared statements:

Test A: Fără Reutilizare (crearea de statement-uri noi)

```
for (int i = 0; i < 1000; i++) {  
    Query query = entityManager.createQuery("SELECT e FROM Employee e WHERE e.id = :id");  
    query.setParameter("id", (long) i);  
    query.getSingleResult();  
}
```

Test B: Cu Reutilizare Statement

```
Query query = entityManager.createQuery("SELECT e FROM Employee e WHERE e.id = :id");  
for (int i = 0; i < 1000; i++) {  
    query.setParameter("id", (long) i);  
    query.getSingleResult();  
}
```

Măsurați și comparați timpii de execuție.

7. Best Practices pentru Conexiunea la Baza de Date

Documentați și implementați:

Securitatea Connection String-ului:

```
# Nu faceți așa (hardcodat în cod)  
connection.url=jdbc:postgresql://localhost:5432/mydb?user=admin&password=secret123  
  
# Faceți așa (variabile de mediu)  
connection.url=jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}  
connection.username=${DB_USER}  
connection.password=${DB_PASSWORD}
```

Configurarea Connection Pool-ului: Documentați setările optime pentru aplicația dvs.:

```
# Dimensiunea pool-ului bazată pe formula: connections = ((core_count * 2) + effective_spindle_count)  
hikari.maximum-pool-size=10
```

```
hikari.minimum-idle=5
hikari.connection-timeout=30000
hikari.idle-timeout=600000
hikari.max-lifetime=1800000
```

Interogări de Validare:

```
hikari.connection-test-query=SELECT 1
```

9. Livrabile

1. **Cod sursă complet** cu toate optimizările implementate
2. **Rezultate măsurări performanță** în format structurat:
 - Comparație problemă N+1 înainte/după
 - Tabel performanță indexare
 - Comparație strategii paginare
 - Statistici cache hit/miss
 - Benchmark-uri operații în masă
3. **Log-uri interogări SQL** arătând impactul optimizării
4. **Output-uri EXPLAIN ANALYZE** pentru interogări cheie
5. **Fișiere de configurare** cu setări optime
6. **Raport cuprinzător (4-5 pagini)** acoperind:
 - Fiecare tehnică de optimizare explicată
 - Măsurători de performanță și analiză
 - Compromisuri și când să folosiți fiecare tehnică
 - Checklist pentru producție
 - Lecții învățate
 - Recomandări pentru îmbunătățiri viitoare

10. Criterii de Evaluare (100 puncte)

- **Rezolvarea Problemei N+1 (15 puncte)**
 - Problemă demonstrată și rezolvată (10)
 - Comparație performanță documentată (5)
- **Indexare (20 puncte)**
 - Selecție corectă de index-uri (10)
 - Măsurători performanță complete (10)
- **Paginare (15 puncte)**
 - Ambele strategii implementate (10)
 - Comparație și analiză (5)
- **Caching (15 puncte)**
 - Implementare cache corectă (10)
 - Invalidare cache funcțională (5)
- **Operații în Masă (10 puncte)**
 - Toate abordările testate și comparate (10)
- **Calitatea Codului (15 puncte)**
 - Best practices urmate (10)

- Externalizare corectă configurare (5)
- **Raport (10 puncte)**
 - Analiză completă cu recomandări (10)

11. Puncte Bonus (+20)

- Implementați caching rezultate interogări la nivel ORM (+5)
- Adăugați monitoring/metrici pentru operații bază de date (+5)
- Implementați endpoint health check bază de date (+3)
- Creați suită testare performanță cu benchmark-uri automate (+7)

Predare

- **Termen limită:** Următoarea sesiune de laborator (14 de zile)